Terraform-Provisioned Serverless Data Preprocessing Pipeline on AWS

William Lorenzo[1,2*]

[1*]Lead Data Analyst/AI Engineer, Interlinked, USA.
[2]Department of Statistics, B.S.: Data Science Concentration, California State University, East Bay, 25800 Carlos Bee Boulevard, Hayward, California, 94542, USA.

Corresponding author(s). E-mail(s): wlorenzo@horizon.csueastbay.edu;

Abstract

Cloud-native architectures have transformed how distributed data systems are provisioned, deployed, and scaled. Infrastructure-as-Code tools support reproducible cloud environments while reducing configuration drift and manual operational overhead. This paper presents the design and implementation of a serverless data preprocessing pipeline on Amazon Web Services provisioned entirely with Terraform. The architecture combines Amazon S3 for storage, AWS Lambda for event-driven preprocessing, IAM roles and policies for least-privilege access control, Lambda layers for dependency management, S3 event notifications for automated execution, and Amazon CloudWatch for observability. When a raw CSV file is uploaded to a designated S3 prefix, a Lambda function is triggered to clean and standardize the dataset using Pandas. The transformed output is written to a processed-data prefix for downstream analytics and machine learning workflows. The pipeline performs column normalization, missing-value handling, removal of low-information features, and type-aware imputation. The resulting system demonstrates how reproducible, auditable, and scalable preprocessing infrastructure can be achieved through a serverless, event-driven design aligned with contemporary MLOps practices.

Keywords: Terraform, AWS Lambda, Amazon S3, serverless computing, Infrastructure-as-Code, data preprocessing, MLOps, CloudWatch, Pandas, ETL

1 Introduction

Cloud-native architectures have fundamentally changed the way modern distributed systems are provisioned and operated. Rather than relying on manual console-based configuration, organizations increasingly define infrastructure declaratively so that environments can be reproduced consistently across development, testing, and production. Among Infrastructure-as-Code frameworks, Terraform is widely used because of its declarative HashiCorp Configuration Language, state management model, provider ecosystem, and execution planning capabilities [1, 2].

At the same time, data-centric applications require reliable preprocessing pipelines to prepare raw data for analytics and machine learning tasks. Manual preprocessing can be difficult to reproduce, error-prone, and poorly suited to event-driven or high-volume environments. This project addresses that challenge by implementing an automated preprocessing pipeline on Amazon Web Services using Terraform to provision all required resources. The system integrates Amazon S3 for object storage, AWS Lambda for serverless transformation, IAM roles and policies for access control, Lambda layers for dependency reuse, S3 event notifications for orchestration, and CloudWatch for runtime logging [3, 5, 7].

The central workflow is event-driven. When a CSV file is uploaded into the `raw/` prefix of an S3 bucket, a Lambda function is invoked automatically to perform Pandas-based preprocessing. The cleaned output is then written to the `processed/` prefix, producing a repeatable AWS-native extraction, transformation, and loading pattern for downstream analytics and machine learning workflows.

To formalize the pipeline, let a raw dataset be represented by a matrix

$$X \in \mathbb{R}^{n \times d}, \tag{1}$$

where n denotes the number of records and d denotes the number of attributes. The preprocessing function is written as

$$\mathcal{P} : X \mapsto \widetilde{X}, \tag{2}$$

where $\widetilde{X}$ is the cleaned dataset produced after normalization, feature filtering, and imputation. In this project, $\mathcal{P}$ is implemented as an event-triggered Lambda transformation executed after object creation in Amazon S3.

2 System Overview

The proposed system is a fully serverless preprocessing pipeline that uses Terraform as the provisioning layer and AWS managed services as the execution environment. The architecture is composed of five main components:

(i) an Amazon S3 bucket partitioned into `raw/` and `processed/` prefixes,
(ii) an AWS Lambda function that performs data cleaning,
(iii) an AWS-managed Lambda layer providing Pandas and related dependencies,
(iv) IAM roles and policies for controlled access to storage and logging, and

(v) S3 event notifications that trigger the transformation workflow automatically.

Figure 1 presents the high-level architecture.

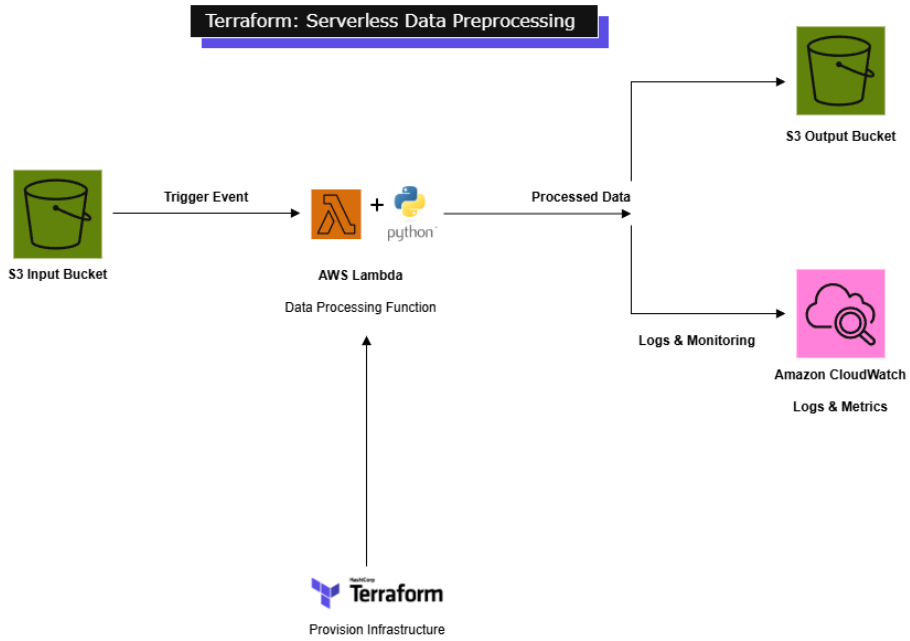

Fig. 1 High-level architecture of the serverless preprocessing pipeline deployed using Terraform. Raw CSV files uploaded to the S3 bucket trigger a Lambda function through S3 event notifications. The Lambda function performs Pandas-based preprocessing and writes outputs to the `processed/` prefix.

Operationally, the workflow proceeds as follows:

1. a user uploads `raw/<filename>.csv` to the provisioned S3 bucket;
2. Amazon S3 emits an `ObjectCreated` event;
3. AWS Lambda receives the event and reads the uploaded object;
4. Pandas-based preprocessing is applied to the dataset;
5. the cleaned dataset is written to `processed/<filename>.csv`; and
6. Amazon CloudWatch records execution logs and transformation metadata.

This event-driven design provides a compact foundation for scalable data ingestion and preprocessing in MLOps-oriented cloud systems.

3 Infrastructure Architecture in Terraform

The entire infrastructure was defined in Terraform HCL, enabling deterministic deployment and lifecycle management. Terraform state provides a canonical mapping between declared resources and provisioned cloud resources, which supports incremental updates, dependency tracking, and drift detection [1].

Terraform planning can be understood as constructing a dependency graph

$$G = (V, E), \tag{3}$$

where V is the set of cloud resources and E is the set of directed dependencies between resources. If an edge $(v_i, v_j) \in E$ exists, then resource v_i must be created or resolved before resource v_j. This graph-based ordering is important for resources such as S3 notifications, which require the Lambda function and invocation permission to exist before the notification trigger is configured.

3.1 S3 data lake structure

The pipeline provisions an S3 bucket named:

```
terraform-pipeline-bucket-wb-20251216
```

Within this bucket, two logical prefixes are used:

$$\texttt{raw/} \rightarrow \text{ingestion zone for unprocessed CSV files,} \tag{4}$$

$$\texttt{processed/} \rightarrow \text{output zone for cleaned CSV files.} \tag{5}$$

Prefix-based organization simplifies event filtering and prevents recursive invocation when the Lambda function writes transformed files back to the same bucket. Only objects created under `raw/` are configured to trigger preprocessing.

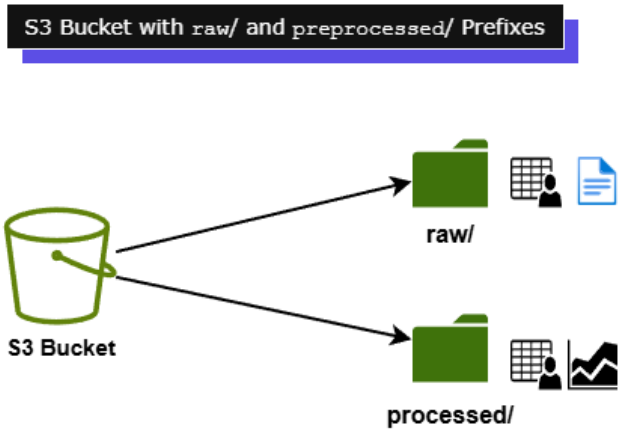

Fig. 2 Amazon S3 bucket organization showing the `raw/` and `processed/` prefixes used for ingestion and transformed outputs.

3.2 Lambda packaging and deployment

The Lambda deployment artifact is constructed through Terraform's `archive_file` data source:

```
data "archive_file" "lambda_zip" {
  type = "zip"
  source_dir = "${path.module}/lambda_code"
  output_path = "${path.module}/lambda.zip"
}
```

This packaging approach ensures that source files in the `lambda_code/` directory are included in the final deployment artifact. Scientific Python dependencies are delegated to an AWS-managed layer instead of being bundled locally, reducing operating-system-specific compatibility problems.

3.3 Lambda function configuration

The Lambda function is configured with the following operational parameters:

- Runtime: Python 3.10
- Architecture: `x86_64`
- Timeout: 60 seconds
- Environment variable: `BUCKET_NAME`
- Execution role with S3 read/write and CloudWatch logging permissions

Pandas and NumPy are provided through an AWS-managed Lambda layer:

```
layers = [
  "arn:aws:lambda:us-east-1:336392948345:layer:AWSSDKPandas-Python310:9"
]
```

Using a managed layer improves compatibility with the Amazon Linux Lambda runtime and reduces packaging complexity [8].

3.4 IAM role and permissions

The Lambda execution role follows least-privilege design principles. It includes the AWS-managed `AWSLambdaBasicExecutionRole` policy and custom permissions for:

- `s3:GetObject`,
- `s3:PutObject`, and
- `s3:ListBucket`.

In addition, the Lambda function requires a resource-based permission that allows S3 to invoke it:

```
action = "lambda:InvokeFunction"
principal = "s3.amazonaws.com"
```

This permission is necessary for S3 event notifications to trigger the Lambda function correctly [3, 5].

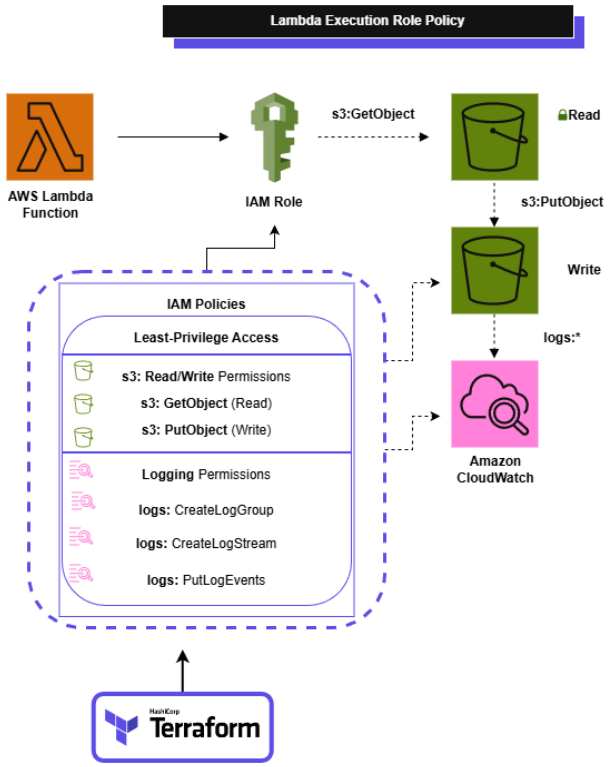

Fig. 3 Lambda execution role and policy structure showing S3 read/write permissions and CloudWatch logging access.

3.5 S3-to-Lambda event notification

Terraform defines the event trigger as follows:

```
resource "aws_s3_bucket_notification" "bucket_notify" {
  bucket = aws_s3_bucket.datalake.id

  lambda_function {
    lambda_function_arn = aws_lambda_function.preprocess.arn
    events = ["s3:ObjectCreated:*"]
    filter_prefix = "raw/"
  }

  depends_on = [
    aws_lambda_function.preprocess,
    aws_lambda_permission.allow_s3
  ]
```

```
14 }
```

This configuration restricts triggering to newly created objects under the `raw/` prefix. The `depends_on` clause enforces correct resource creation order and helps prevent notification failures.

4 Preprocessing Methodology

The preprocessing logic is implemented inside the Lambda handler so that transformation occurs at invocation time. This design avoids module-level side effects during runtime initialization and keeps execution tightly coupled to each object event.

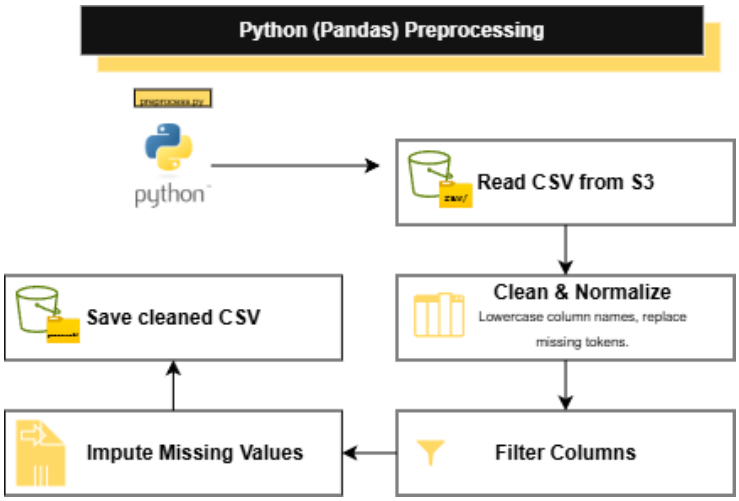

Fig. 4 Illustrative view of the Pandas-based preprocessing workflow implemented within the Lambda function.

4.1 Loading and parsing

After invocation, the Lambda function retrieves the target CSV object from S3, decodes it safely as UTF-8, and loads it into a Pandas DataFrame. Initial dimensions are logged to CloudWatch to preserve execution visibility.

4.2 Cleaning and normalization

Column names are normalized by lowercasing and standardizing whitespace or formatting conventions. Common missing-value tokens are represented as the set

$$\mathcal{M} = \{\texttt{""}, \texttt{"na"}, \texttt{"nan"}, \texttt{"none"}, \texttt{"null"}, \texttt{"error"}, \texttt{"warn"}\}. \tag{6}$$

Each token $m \in \mathcal{M}$ is mapped to a null representation prior to imputation. This normalization step improves schema consistency and downstream model compatibility.

4.3 Feature filtering

Columns with excessive missingness are removed according to the rule

$$\frac{1}{n}\sum_{i=1}^{n}\mathbf{1}\{x_{ij}\text{ is missing}\} > 0.80, \tag{7}$$

where feature j is dropped if more than 80% of its entries are absent. Single-valued columns are also removed because they provide negligible information for downstream analysis.

4.4 Missing-value imputation

Let X_j denote the jth feature. The imputation strategy is type-aware:

$$\widehat{x}_{ij} = \begin{cases} \text{median}(X_j), & \text{if } X_j \text{ is numeric and } x_{ij} \text{ is missing,} \\ \text{mode}(X_j), & \text{if } X_j \text{ is categorical and a mode exists,} \\ \texttt{Unknown}, & \text{otherwise.} \end{cases} \tag{8}$$

This rule ensures that the cleaned dataset contains no unresolved null values, improving compatibility with downstream analytics and machine learning algorithms [9, 10].

4.5 Writing output and observability

After preprocessing, the transformed DataFrame is serialized and written to:

```
processed/<filename>.csv
```

CloudWatch logs record dataset dimensions, transformation metadata, and confirmation of successful output creation [7].

5 Algorithmic Summary

The overall transformation logic is summarized in Algorithm 1.

Algorithm 1 Event-driven preprocessing pipeline

Require: S3 object event e
Ensure: Cleaned CSV written to `processed/`

Extract bucket name and object key from e
Read raw CSV from `raw/` into DataFrame X
Normalize column names and missing-value tokens
for each feature X_j in X **do**
    if missingness$(X_j) > 0.80$ **then**
        Drop X_j
    else if X_j has only one unique value **then**
        Drop X_j
    else if X_j is numeric **then**
        Impute missing values with median(X_j)
    else
        Impute missing values with mode(X_j) or `Unknown`
    end if
end for
Write cleaned DataFrame $\widetilde{X}$ to `processed/`
Log execution metadata to CloudWatch

6 Results and Final System Behavior

From an infrastructure perspective, the project demonstrates that Terraform can provision a complete serverless preprocessing workflow with minimal manual intervention. From an operational perspective, the architecture exhibits four desirable properties.

First, the system is reproducible because infrastructure is declared in version-controlled HCL rather than configured manually. Second, it is scalable because Lambda execution is event-driven and does not require persistent servers. Third, it is auditable because Terraform state, IAM policies, and CloudWatch logs expose both deployment and runtime behavior. Fourth, it is extensible because additional preprocessing logic, validation rules, or downstream integrations can be layered into the same event-driven pattern.

The realized behavior of the pipeline is summarized below:

1. a raw CSV upload to the `raw/` prefix initiates processing automatically;
2. the Lambda function loads and transforms the dataset using Pandas;
3. the output is written to the `processed/` prefix without recursive retriggering; and
4. logs and transformation traces are preserved in CloudWatch for debugging and monitoring.

These characteristics make the architecture a strong baseline for cloud-native preprocessing in analytics and machine learning systems.

7 Discussion

This project illustrates the practical convergence of Infrastructure-as-Code, serverless computing, and data engineering. By combining Terraform with AWS managed services, the pipeline replaces manual preprocessing workflows with a reproducible and event-responsive cloud design. The use of S3 prefix partitioning, Lambda layers, and least-privilege IAM policies contributes to both maintainability and operational safety.

A key limitation is that the current pipeline is specialized for CSV-based tabular preprocessing. More complex workloads, such as schema evolution, multi-file joins, streaming enrichment, or large-scale transformations, may require orchestration services such as AWS Step Functions, AWS Glue, or containerized execution platforms. Another limitation is that the present implementation applies rule-based preprocessing rather than domain-adaptive or learned transformation policies.

Nevertheless, the architecture provides an effective starting point for MLOps pipelines in which clean, consistent, and automatically generated datasets are required before model training or inference.

8 Conclusion

This paper presented a Terraform-provisioned serverless data preprocessing pipeline on AWS. The system uses Amazon S3 for storage, AWS Lambda for event-triggered cleaning, IAM for access control, Lambda layers for dependency compatibility, and CloudWatch for observability. By expressing infrastructure declaratively and preprocessing logic programmatically, the pipeline achieves reproducibility, scalability, and reduced operational overhead.

More broadly, the project demonstrates how serverless event-driven architectures can serve as robust building blocks for automated preprocessing in modern data and machine learning systems. Future work may extend the system with schema validation, data quality scoring, multi-format ingestion, workflow orchestration, and direct integration with downstream model training services.

Supplementary information. Supplementary material may include the full Terraform configuration files, AWS Lambda source code, architecture diagrams, and example CloudWatch logs.

Acknowledgements. The author thanks Interlinked for providing a professional context in which cloud-native data engineering and AI infrastructure can be explored in practice.

Declarations

Funding
No external funding was reported for this study.

Conflict of interest/Competing interests
The author declares no competing interests.

Ethics approval and consent to participate

Not applicable. This project does not involve human subjects, animal subjects, or identifiable private information.

Consent for publication

Not applicable.

Data availability

No public dataset is formally archived with this manuscript. Example pipeline inputs are CSV files uploaded by the user to the configured Amazon S3 bucket.

Materials availability

Terraform configuration files and Lambda source code can be shared in an accompanying source repository.

Code availability

The Terraform and Python source code used to provision and execute the pipeline are included in Appendix B.

Author contribution

William Lorenzo conceived the project, implemented the Terraform infrastructure and Lambda preprocessing workflow, and wrote the manuscript.

Appendix A AWS Lambda Preprocessing Source Code

The preprocessing workflow was implemented as an AWS Lambda function written in Python using Pandas and NumPy. The function is triggered automatically through Amazon S3 event notifications whenever a CSV object is uploaded into the `raw/` prefix of the provisioned data lake bucket.

Listing 1 AWS Lambda preprocessing implementation in preprocess.py

```python
import boto3
import pandas as pd
import numpy as np
import io
import logging

logger = logging.getLogger()
logger.setLevel(logging.INFO)

def lambda_handler(event, context):
    s3 = boto3.client("s3")

    # Extract bucket and key from event
    record = event["Records"][0]
    bucket = record["s3"]["bucket"]["name"]
    key = record["s3"]["object"]["key"]

    logger.info(f"Processing: s3://{bucket}/{key}")

    # Read file from S3
```

```
    obj = s3.get_object(Bucket=bucket, Key=key)
    csv_data = obj["Body"].read().decode("utf-8", errors="ignore")

    df = pd.read_csv(io.StringIO(csv_data))
    logger.info(f"Original shape: {df.shape}")

    # Clean column names
    df.columns = df.columns.str.lower().str.strip()

    # Replace common missing values
    missing_strings = ["", "na", "nan", "none", "null", "error", "warn"]
    df = df.replace(missing_strings, np.nan)

    # Drop columns with greater than 80 percent missing values
    missing_fraction = df.isna().mean()
    drop_cols = missing_fraction[missing_fraction > 0.8].index.tolist()

    logger.info(f"Dropping columns: {drop_cols}")
    df = df.drop(columns=drop_cols)

    # Fill missing values
    for col in df.columns:
        if df[col].dtype == "object":
            mode = df[col].mode()
            fill_val = mode.iloc[0] if not mode.empty else "unknown"
            df[col] = df[col].fillna(fill_val)
        else:
            median = df[col].median()
            df[col] = df[col].fillna(median)

    logger.info(f"Final shape: {df.shape}")

    # Write cleaned CSV to processed prefix
    output = io.StringIO()
    df.to_csv(output, index=False)

    processed_key = key.replace("raw/", "processed/")
    if processed_key == key:
        processed_key = f"processed/{key.split('/')[-1]}"

    logger.info(f"Saving cleaned file to: s3://{bucket}/{processed_key}")

    s3.put_object(
        Bucket=bucket,
        Key=processed_key,
        Body=output.getvalue().encode("utf-8")
    )

    return {
        "statusCode": 200,
```

```
        "message": "Success"
    }
```

Appendix B Terraform Infrastructure Configuration

The AWS infrastructure was provisioned entirely using Terraform. The following appendices contain the principal Infrastructure-as-Code configuration files used for deployment.

B.1 Terraform Provider and S3 Configuration

Listing 2 Terraform provider and S3 bucket configuration in main.tf

```
provider "aws" {
  profile = "William"
  region = "us-east-1"
}

resource "aws_s3_bucket" "datalake" {
  bucket = var.bucket_name
  force_destroy = true

  tags = {
    Name = "Terraform-DS-Pipeline"
  }
}

resource "aws_s3_object" "folders" {
  for_each = toset(["raw/", "processed/"])

  bucket = aws_s3_bucket.datalake.bucket
  key = each.value
}

terraform {
  required_providers {
    archive = {
      source = "hashicorp/archive"
      version = "~> 2.0"
    }

    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

B.2 IAM Configuration

Listing 3 Terraform IAM role and policy configuration in iam.tf

```
resource "aws_iam_role" "lambda_role" {
  name = "ds-lambda-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"

    Statement = [{
      Action = "sts:AssumeRole"

      Principal = {
        Service = "lambda.amazonaws.com"
      }

      Effect = "Allow"
    }]
  })
}

resource "aws_iam_role_policy_attachment" "lambda_basic_execution" {
  role = aws_iam_role.lambda_role.name
  policy_arn = "arn:aws:iam::aws:policy/service-role/
      AWSLambdaBasicExecutionRole"
}

resource "aws_iam_role_policy" "lambda_s3_policy" {
  role = aws_iam_role.lambda_role.id

  policy = jsonencode({
    Version = "2012-10-17"

    Statement = [
      {
        Effect = "Allow"
        Action = "s3:ListBucket"
        Resource = aws_s3_bucket.datalake.arn
      },
      {
        Effect = "Allow"
        Action = [
          "s3:GetObject",
          "s3:PutObject"
        ]
        Resource = "${aws_s3_bucket.datalake.arn}/*"
      }
    ]
  })
```

```
}
```

B.3 Lambda Deployment Configuration

Listing 4 Terraform Lambda deployment and event notification configuration in lambda.tf

```
data "archive_file" "lambda_zip" {
  type = "zip"
  source_dir = "${path.module}/lambda_code"
  output_path = "${path.module}/lambda.zip"
}

resource "aws_lambda_function" "preprocess" {
  function_name = "ds_preprocess_lambda"
  runtime = "python3.10"
  handler = "preprocess.lambda_handler"
  architectures = ["x86_64"]

  role = aws_iam_role.lambda_role.arn
  filename = data.archive_file.lambda_zip.output_path
  source_code_hash = filebase64sha256(data.archive_file.lambda_zip.output_path
      )

  timeout = 60
  publish = false

  environment {
    variables = {
      BUCKET_NAME = var.bucket_name
    }
  }

  layers = [
    "arn:aws:lambda:us-east-1:336392948345:layer:AWSSDKPandas-Python310:9"
  ]
}

resource "aws_lambda_permission" "allow_s3" {
  statement_id = "AllowExecutionFromS3"
  action = "lambda:InvokeFunction"
  function_name = aws_lambda_function.preprocess.function_name
  principal = "s3.amazonaws.com"
  source_arn = aws_s3_bucket.datalake.arn
}

resource "aws_s3_bucket_notification" "bucket_notify" {
  bucket = aws_s3_bucket.datalake.id

  lambda_function {
```

```
    lambda_function_arn = aws_lambda_function.preprocess.arn
    events = ["s3:ObjectCreated:*"]
    filter_prefix = "raw/"
  }

  depends_on = [
    aws_lambda_function.preprocess,
    aws_lambda_permission.allow_s3
  ]
}
```

B.4 Terraform Variables

Listing 5 Terraform variable definitions in variables.tf

```
variable "region" {
  default = "us-east-1"
}

variable "bucket_name" {
  description = "Name of the S3 bucket to hold the data lake"
}
```

B.5 Terraform Outputs

Listing 6 Terraform output definitions in outputs.tf

```
output "bucket" {
  value = aws_s3_bucket.datalake.bucket
}

output "lambda_name" {
  value = aws_lambda_function.preprocess.function_name
}
```

References

[1] HashiCorp. Terraform documentation. Available at: https://developer.hashicorp.com/terraform/docs

[2] HashiCorp. What is Infrastructure as Code with Terraform? Available at: https://developer.hashicorp.com/terraform/tutorials/aws-get-started/infrastructure-as-code

[3] Amazon Web Services. AWS Lambda developer guide. Available at: https://docs.aws.amazon.com/lambda/latest/dg/welcome.html

[4] Amazon Web Services. Amazon S3 user guide. Available at: https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html

[5] Amazon Web Services. Amazon S3 event notifications. Available at: https://docs.aws.amazon.com/AmazonS3/latest/userguide/EventNotifications.html

[6] Amazon Web Services. AWS Identity and Access Management user guide. Available at: https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html

[7] Amazon Web Services. Amazon CloudWatch user guide. Available at: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html

[8] Amazon Web Services. AWS SDK for pandas documentation. Available at: https://aws-sdk-pandas.readthedocs.io/

[9] Pandas Development Team. Pandas documentation. Available at: https://pandas.pydata.org/docs/

[10] McKinney W. *Python for Data Analysis*. 3rd ed. Sebastopol, CA: O'Reilly Media; 2022.

[11] Burns B. *Designing Distributed Systems*. Sebastopol, CA: O'Reilly Media; 2018.

[12] Kleppmann M. *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media; 2017.